Centre universitari adscrit a la

Universitat Pompeu Fabra Barcelona

Bachelor in Computer Engineering — Management and Information Systems

Ethical Pentesting in Virtualized Environments with EVE-NG

Memory

Volume I

Eloi Egea Rada
Supervisor: Pere Vidiella i Catalan

April 2026

Contents

List of Tables

1 Introduction

Cybersecurity education faces a critical challenge: the disconnect between theoretical knowledge of IT systems and practical understanding of how security failures manifest in integrated environments. While students in the Computer Engineering degree program learn fundamental concepts—databases, network protocols, operating system internals, and software architecture—they rarely experience how misconfigurations and design flaws in these domains directly translate to security vulnerabilities.

Current training approaches amplify this gap. Commercial platforms such as HackTheBox**hackthebox** and TryHackMe**tryhackme** focus on isolated challenge-based scenarios that abstract away system complexity. Professional pentesting certifications (e.g., CEH, OSCP**ceh_framework**, **oscp_exam**) expect hands-on experience in realistic environments, yet many academic programs struggle to provide such environments due to cost, complexity, or vendor lock-in.

Within the TecnoCampus Bachelor's Degree in Computer Engineering (Management and Information Systems track), the "Introduction to Cybersecurity" course currently lacks a structured, reusable, and automatable lab environment that:

- Aligns pentesting scenarios with specific degree learning outcomes
- Enables students to recognize vulnerabilities within integrated systems (not isolated exploits)
- Provides reproducible environments for consistent assessment
- Reduces instructor workload through automation and documentation
- Remains cost-effective and institution-independent (open-source, locally maintained)

2 State of the Art

Cybersecurity education has experienced exponential growth over the past decade, driven by increasing industry demand for skilled professionals and the accessibility of online learning platforms. However, this landscape presents both opportunities and challenges: while students have unprecedented access to training resources, the fragmentation of educational approaches has created gaps between abstract theoretical knowledge and practical, integrated system security. This chapter examines existing educational platforms and methodologies, identifies gaps in current offerings, and positions this bachelor's thesis within the broader context of cybersecurity pedagogical innovation.

2.1 Existing Laboratory and Training Platforms

The following section provides a comprehensive overview of major platforms used for cybersecurity education and practical training.

2.1.1 HackTheBox

Overview: HackTheBox **hackthebox** is a prominent online platform offering Capture-the-Flag (CTF) challenges and virtual machines designed for hands-on penetration testing practice. The platform provides a large collection of intentionally vulnerable systems with varying difficulty levels.

Key Characteristics:

- **Challenge-based approach**: Individual machines or challenges designed to teach specific exploitation techniques
- **Community-driven**: Thousands of pre-built challenges from the security community

- **Difficulty levels:** Machines categorized by skill requirement (easy, medium, hard, insane)
- **Free and Pro tiers:** Basic access available without cost; premium features for advanced content
- **Isolated scenarios:** Each challenge focuses on a specific technique or vulnerability type

Strengths:

- Massive library of pre-built challenges requiring minimal setup
- Strong community and competitive leaderboards encourage engagement
- Industry recognition: many professional certifications reference HackTheBox as supplementary training
- Variety of attack vectors and operating systems

Limitations:

- Challenges are isolated; limited networking between systems
- No formal curriculum mapping to specific degree programs
- Scenarios often abstract away real-world complexity
- Platform dependency: students cannot deploy labs locally
- Limited automation or integration with institutional learning systems
- Content is not institution-controlled, making it difficult to align with specific curricula

2.1.2 TryHackMe

Overview: TryHackMe **tryhackme** is a gamified cybersecurity training platform emphasizing beginner-friendly content with structured learning paths. The platform combines interactive tutorials, guided labs, and challenges within a narrative-driven framework.

Key Characteristics:

- **Gamification:** Points, levels, and achievements to motivate learners
- **Structured learning paths:** Predefined sequences of rooms (lessons) organized by skill and topic
- **Interactive tutorials:** Embedded guidance and step-by-step walkthroughs
- **Browser-based labs:** No local installation required
- **Community and Discord:** Peer support and instructor interaction

Strengths:

- Excellent for beginners; low barrier to entry
- Gamification increases student engagement and completion rates
- Structured learning paths provide clear progression
- Active community support
- Diverse content covering defensive and offensive skills

Limitations:

- Platform-dependent; cannot be deployed on institutional infrastructure

- Free tier is heavily limited; premium subscription required for advanced content
- Simplified scenarios often omit real-world system complexity
- Content is externally managed; no customization for specific curricula
- Scenarios are designed to be completable in short timeframes, potentially sacrificing depth
- Limited focus on integrated network penetration testing

2.1.3 Cisco Networking Academy

Overview: Cisco Networking Academy **cisco_netacad** is a comprehensive, institution-based cybersecurity training program integrated with formal curricula worldwide. The program offers certifications (CCNA, CyberOps) and structured coursework.

Key Characteristics:

- **Corporate partnership:** Official Cisco curriculum with institutional support
- **Formal certifications:** Aligned with professional certifications (CCNA)
- **Network simulation:** Cisco Packet Tracer for virtual network design
- **Comprehensive curriculum:** From networking fundamentals to security
- **Instructor-led delivery:** Typically offered through educational institutions

Strengths:

- Strong institutional support and curriculum design

- Formal recognition through industry certifications
- Network simulation tools (Packet Tracer) are powerful and realistic
- Integration with degree programs
- Vendor-backed expertise and resources

Limitations:

- Primary focus is networking, not penetration testing
- CyberOps certification is defensive-oriented, not offensive security
- Limited hands-on exploitation and penetration testing content
- Vendor lock-in to Cisco technologies
- Packet Tracer is not suitable for full operating system simulation or security tool integration
- Limited support for integrated penetration testing workflows

2.1.4 SEED Labs

Overview: SEED (Security, Privacy, and Trust Labs) is an open-source collection of hands-on security exercises designed by researchers for academic cybersecurity education. Each lab focuses on specific security concepts (buffer overflows, SQL injection, cryptography, etc.). **seedlabs**

Key Characteristics:

- **Research-backed design:** Developed by cybersecurity researchers at universities
- **Docker-based deployment:** Portability and reproducibility through containerization
- **Open-source:** Freely available; institutions can deploy locally

- **Concept-focused:** Each lab teaches a specific security principle
- **Comprehensive documentation:** Detailed handouts and guidelines for instructors

Strengths:

- Highly regarded in academic communities; research-validated pedagogical approaches
- Docker-based design enables easy institutional deployment
- Open-source ensures no vendor lock-in
- Labs cover deep technical concepts (memory safety, cryptographic protocols)
- Excellent documentation and instructor support

Limitations:

- Each lab is conceptually isolated; limited networking or integration between labs
- No structured penetration testing workflows (reconnaissance $\rightarrow$ exploitation $\rightarrow$ escalation)
- Requires significant instructor customization for curriculum integration
- Limited automation for lab deployment and reset
- Focus on security concepts rather than realistic multi-system environments
- Not designed for practicing complete penetration testing methodologies

2.1.5 Game of Active Directory (GOAD)

Overview: GOAD **goad** is an open-source project providing intentionally vulnerable Active Directory environments designed specifically for penetration testing practice. It simulates a realistic corporate Windows domain infrastructure with multiple misconfigurations and vulnerabilities.

Key Characteristics:

- **AD-focused:** Entire lab built around Windows Active Directory
- **Terraform/Vagrant-based:** Infrastructure-as-Code for reproducible deployment
- **Realistic domain structure:** Simulates real corporate Active Directory environments
- **Multiple exploitation paths:** Designed to teach lateral movement and privilege escalation
- **Community-maintained:** Open-source with active development

Strengths:

- Exceptional for Windows-focused penetration testing training
- Highly realistic Active Directory scenarios
- Excellent for teaching privilege escalation on Windows domains
- Infrastructure-as-Code approach enables reproducibility
- Active community and maintained codebase

Limitations:

- Narrow focus on Windows AD; limited coverage of other platforms

- Does not cover reconnaissance, web application security, or network analysis
- Steep learning curve for infrastructure setup and maintenance
- Not designed as a comprehensive penetration testing progression
- Limited instructor resources or formal curriculum

2.1.6 VulnHub

Overview: VulnHub **vulnhub** is a community-driven repository of vulnerable virtual machines designed for penetration testing practice. Users download complete VM images and deploy them locally in their own environments.

Key Characteristics:

- **Community-created:** Machines contributed by security practitioners
- **Downloadable VMs:** Complete VM images for local deployment
- **Diverse difficulty:** Machines range from beginner to advanced
- **Topic-specific:** Machines often focus on specific techniques or vulnerabilities
- **Open-ended challenges:** Minimal guidance; students must determine objectives

Strengths:

- Excellent for self-directed learners who want complete autonomy
- Machines are contributed by practicing security professionals, reflecting real-world scenarios
- Local deployment avoids platform dependencies

- Free and open resource
- Teaches problem-solving and methodology development

Limitations:

- Minimal formal curriculum structure; students must self-organize learning
- Quality and consistency of machines varies significantly
- No built-in progression or guided learning paths
- Documentation is often minimal or community-driven
- Difficult to integrate into institutional courses without significant instructor work
- No automation for deployment, reset, or assessment
- Isolated machines; limited network topology complexity

2.1.7 Hack4u Academy

Overview: Hack4u Academy**hack4u** is a structured online cybersecurity training platform offering comprehensive penetration testing courses. It combines video instruction, practical labs, and community support with emphasis on methodology over tool memorization.

Key Characteristics:

- **Methodology-driven:** Emphasizes PTES (Penetration Testing Execution Standard) and systematic approaches
- **Progressive curriculum:** Structured from beginner to advanced levels
- **Hands-on labs:** Integrated practical exercises within video-based instruction
- **Video content:** Detailed walkthroughs and demonstrations

- **Community support:** Discussion forums and peer learning

Strengths:

- Strong emphasis on methodology and systematic pentesting approaches
- Well-structured curriculum with clear learning progression
- Active instructor engagement and community support
- Balanced approach: teaches both tools and underlying concepts
- Accessible to practitioners at various skill levels

Limitations:

- Subscription-based; not free or open-source
- Platform-dependent; cannot be deployed institutionally
- Limited formal integration with academic curricula
- Individual user focus rather than classroom-oriented design
- Not designed for automated assessment or large-scale institutional deployment

2.2 Comparative Platform Analysis Matrix

The following table synthesizes key characteristics across platforms:

Platform	Approach	Local Deploy	Curriculum	Automation	Network
HackTheBox	CTF/Challenge	No	Minimal	No	Isolated
TryHackMe	Gamified	No	Structured	No	Limited
Cisco Academy	Academic	Yes (Packet Tracer)	Strong	Limited	Network-focused
SEED Labs	Concept-focused	Yes (Docker)	Concept-based	Minimal	Isolated
GOAD	AD-focused	Yes (Terraform)	None	Moderate	AD Domain
VulnHub	Community VMs	Yes	None	No	Isolated
Hack4u Academy	Methodology	External	Strong	Minimal	Limited

Table 2.1: Comparative Analysis: Existing Cybersecurity Training Platforms

2.3 Security Frameworks and Methodologies

Beyond specific platforms, the field of cybersecurity education is informed by several authoritative frameworks and standards that guide both attack methodologies and defensive best practices.

2.3.1 OWASP Top 10

The OWASP (Open Web Application Security Project) Top 10 is a regularly updated list of the most critical security vulnerabilities in web applications **owasp2021**. For educational purposes, the Top 10 provides a focused curriculum for teaching web application security without requiring exhaustive coverage of all possible vulnerabilities.

Relevance to bachelor's thesis: Lab 2 (Web Application Vulnerabilities) is directly aligned with OWASP Top 10 categories, ensuring students learn vulnerabilities that represent real-world attack vectors.

2.3.2 CIS Critical Controls

The Center for Internet Security (CIS) publishes a prioritized set of security controls (currently version 8.0) representing the most effective defensive techniques **cis_controls**. These controls span network defense, data protection, and incident response.

Relevance to bachelor's thesis: Lab 1 (Reconnaissance) and Lab 4 (Privilege Escalation) incorporate defensive controls, teaching students not only how to attack but why specific security measures are implemented.

2.3.3 NIST Cybersecurity Framework

The National Institute of Standards and Technology (NIST) provides a framework organizing cybersecurity activities into five functions: Identify, Protect, Detect, Respond, Recover. This framework is widely adopted by organizations and government agencies **nist_framework**.

Relevance to bachelor's thesis: The bachelor's thesis project itself, as described in Chapter 1, adheres to NIST principles by designing a framework (EVE-NG labs) that enables institutions to Identify vulnerabilities (through labs) and Protect systems (through remediation understanding).

2.4 Pedagogical Approaches in Cybersecurity Education

Beyond specific platforms, several pedagogical philosophies inform effective cybersecurity training:

2.4.1 Active Learning and Hands-On Practice

Research in cybersecurity education consistently demonstrates that hands-on, problem-based learning is superior to lecture-based instruction for developing practical skills. Bloom's taxonomy sug-

gests that students must progress from knowledge comprehension through application to analysis and synthesis. Pentesting labs facilitate this progression: students begin by understanding reconnaissance techniques (knowledge), apply them to specific targets (application), analyze results to infer system architecture (analysis), and ultimately synthesize complete penetration testing workflows.

2.4.2 Realistic System Complexity

Effective cybersecurity training must present students with systems of sufficient complexity to simulate real-world scenarios. Isolated, single-service laboratories teach tool usage but fail to prepare students for integrated environments where reconnaissance insights directly inform exploitation strategy, and initial compromise enables lateral movement.

2.4.3 Automation and Reproducibility

For training to be scalable and fair, identical environments must be reliably deployable. Automated deployment and reset mechanisms reduce instructor overhead and ensure consistent student experiences across multiple offering of a course.

2.5 Gap Analysis: What Existing Solutions Miss

Despite the proliferation of cybersecurity training platforms, significant gaps remain:

Gap 1: Curriculum Integration

Most platforms are designed for independent learners or certification prep, not for integration into specific degree programs. No major platform explicitly aligns pentesting scenarios with the learning outcomes of a particular computer science curriculum.

Gap 2: Institution Control

Effective educational integration requires that institutions main-

tain control over content, deployment, and assessment. External platforms introduce dependencies and limit customization. Conversely, platforms requiring significant institutional development (Docker composition, lab orchestration) impose high maintenance burdens.

Gap 3: Integrated Network Penetration Testing

Most platforms focus on isolated services or individual techniques. Few provide realistic, integrated network topologies where reconnaissance informs the entire attack chain, lateral movement is necessary, and privilege escalation varies by system.

Gap 4: Automation and Scalability

Even platforms designed for classroom use often lack automated deployment, reset, and assessment mechanisms. This limitation makes large-scale institutional adoption difficult and increases instructor workload.

Gap 5: Balance of Guidance and Autonomy

Effective labs must strike a balance: sufficient structure to ensure students focus on intended learning outcomes, but sufficient openness to encourage problem-solving and methodology development.

2.6 The Need for EVE-NG Based Labs

This state of the art analysis reveals that while excellent individual components exist—gamified learning (TryHackMe), challenging scenarios (HackTheBox), research-backed content (SEED Labs), specialized domains (GOAD)—no existing solution comprehensively addresses the needs of a university cybersecurity education program:

- **Curriculum-aligned labs**: Mapped to specific degree learning outcomes
- **Integrated network scenarios**: Multi-system penetration testing with realistic complexity

- **Institutional control:** Deployable locally; customizable; maintainable by university staff
- **Complete automation:** One-command deployment and reset; built-in assessment
- **Openness and flexibility:** EVE-NG's platform-agnostic design enables future adaptation
- **System-level realism:** EVE-NG enables full virtual machine workflows (real OS images, ISO-based installation, and native kernel behavior), which is essential for reproducing the same technical conditions used in VMware/VirtualBox-based pentesting practice

Although Containerlab**containerlab** can be valuable for container-based network emulation, it is not a direct comparison target for this project. The objective of this bachelor's thesis is to build integrated pentesting labs that operate at full system depth (host OS, services, and kernel-level behavior), not only at containerized service level.

The bachelor's thesis project bridges this gap by leveraging EVE-NG**eve_ng** (a platform combining network simulation capabilities with institutional independence) to create a comprehensive, reusable, automatable teaching package for cybersecurity practical labs. This solution synthesizes the pedagogical strengths of existing platforms while addressing their limitations through institutional control, curriculum alignment, and automation.

3 Objectives and Scope

3.1 Main Objective

Develop, implement, and validate a reusable teaching package of practical cybersecurity labs based on EVE-NG**eve_ng**, enabling students in the "Introduction to Cybersecurity" course to acquire practical skills in ethical pentesting through realistic and automated scenarios.

3.2 Specific Objectives (Measurable with KPIs)

3.2.1 OE1: Design and Implement EVE-NG Labs

Description: Create four functional and interconnected thematic labs. **Measurable KPIs:**

- 4 fully functional .unl topologies (100%)
- Deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Compatibility with EVE-NG Community and Professional editions

Deliverables:

- 4 .unl topology files (Lab 1: implemented; Labs 2–4: planned)
- 8-12 optimized virtual machine images
- Technical configuration documentation for each lab

OE2: Develop Automation Scripts

Description: Create a complete automation system for the lab lifecycle.

Measurable KPIs:

- 12 functional scripts (3 per lab: deploy, reset, validate)
- Reset execution time $\leq$ 30 seconds per lab
- Automatic validation coverage $\geq$ 80% of critical components
- Zero errors in 10 consecutive executions of the complete cycle

Deliverables:

- Deployment scripts (Bash/Python) — 3 per lab: deploy, reset, validate
- Automated reset scripts
- Validation and status verification scripts

OE3: Develop Structured Teaching Material

Description: Develop comprehensive documentation for students and teachers.

Measurable KPIs:

- 4 user manuals (1 per lab), minimum 15 pages each
- 4 assessment rubrics with quantifiable criteria
- 1 teacher guide with deployment instructions
- Initial setup time per teacher $\leq$ 1 hour

Deliverables:

- Student manual (English, markdown/PDF format)
- Teacher guide with detailed instructions
- Structured assessment rubrics
- Technical README with requirements and instructions

OE4: Validate Usability and Educational Effectiveness

Description: Verify that the package meets educational and technical requirements.

Measurable KPIs:

- Average resolution time per lab: 90-120 minutes
- Successful completion rate $\geq$ 85% (pilot with 10 users)
- User satisfaction score $\geq$ 4/5
- Zero critical incidents in production environment

Deliverables:

- Pilot validation report
- Usability and performance metrics
- Implemented improvement recommendations

3.3 Definition of the Client and Final User

3.4 Principal Client

- **Responsible professor:** Pere Vidiella i Catalan
- **Course:** Introduction to Cybersecurity (GEISI, Tecnocampus)
- **Context:** Undergraduate teaching in practical cybersecurity

3.5 Primary End Users

- **GEISI Students:** 25-30 students per edition
- **Profile:** Basic knowledge of networks and operating systems
- **Objective:** Learn ethical pentesting in a practical and safe manner

3.6 Secondary End Users

- **Cybersecurity Faculty:** Other teachers interested in reusing the material
- **Postgraduate Students:** Possible extensions of the material for advanced levels

3.7 Potential Audience

3.8 Internal Scope (Tecnocampus)

- Other courses related to cybersecurity
- Research projects in computer security
- Continuing education and professional certifications

3.9 External Scope

- Educational institutions with training in cybersecurity
- Specialized vocational training centers
- Companies with internal training programs in security

3.10 Scope Boundaries

Included in this bachelor's thesis:

- Design and implementation of 4 pentesting labs in EVE-NG
- Integration of real vulnerabilities and techniques aligned with OWASP Top 10**owasp2021** and CIS Critical Controls**cis_controls**
- Automation scripts for deployment, validation, and lab reset
- Complete technical documentation for instructors and future maintainers

- Student guides and learning objectives for each lab
- Assessment rubrics for evaluating student lab completion

Explicitly NOT included (out of scope):

- Integration with third-party Learning Management Systems (LMS) or Moodle
- Development of proprietary lab management software or web interfaces
- Creation of automated grading systems beyond basic script validation
- Commercial licensing or distribution models
- Formal pedagogical evaluation studies (e.g., pre/post learning assessments)
- Creation of labs beyond the four specified domains

3.11 KPIs and Performance Indicators

3.11.1 Technical Indicators

Metric	Objective	Measurement Method
Lab deployment time	$\leq$ 2 min	Automated timing
VM boot success rate	$\geq$ 95%	System logs + validation scripts
Full reset time	$\leq$ 30 seg	Scripts with timestamps
Scenario reproducibility	100%	Automated tests

Table 3.1: Technical key performance indicators for EVE-NG lab deployment

3.11.2 Educational Indicators

3.11.3 Quality Indicators

Note: All KPIs will be measured during the pilot validation phase (April-May 2026) and documented in the final bachelor's thesis re-

Metric	Objective	Measurement Method
Lab resolution time	90-120 min	Time tracking during pilot
Successful completion rate	$\geq$ 85%	User progress tracking during pilot
User satisfaction	$\geq$ 4/5	Post-use survey
Key concept understanding	$\geq$ 80% correct	Assessment questionnaire

Table 3.2: Educational key performance indicators for lab effectiveness

Metric	Objective	Measurement Method
Documentation coverage	100% of features	Verification checklist
Critical errors	0	Exhaustive testing
EVE-NG version compatibility	100%	Tests in multiple environments
Interface usability	$\geq$ 4/5	UX heuristic evaluation

Table 3.3: Quality key performance indicators for lab package

port.

4 Methodology

This chapter outlines the comprehensive methodology employed in this bachelor's thesis project, encompassing research strategy, development approach, validation methods, and project lifecycle management. The project integrates principles from software engineering, cybersecurity pedagogy, and empirical research to ensure systematic development and validation of the EVE-NG**eve_ng**-based teaching package.

4.1 Research and Development Phases

The project is structured in four distinct phases, each with specific deliverables, milestones, and validation checkpoints:

4.1.1 Phase 0: Preparation and Infrastructure (July-November 2025)

Objective: Establish the technical foundation and personal capacity for the project.

Activities:

- HomeLab infrastructure initialization (Proxmox**proxmox**, networking, storage)
- Development environment setup (VSCode**vscode**, LaTeX**latex_project**, Git)
- Security tools installation (Kali Linux**kali_linux**, Metasploit**metasploit**, Burp Suite**burp_suite**)
- Technical documentation and baseline establishment

Deliverables:

- Fully operational virtualization infrastructure

- Development environment configuration
- Baseline documentation of system architecture

Duration: 105 hours | **Status**: 100% completed

4.1.2 Phase 1: Avantprojecte and Conceptual Design (October 2025 - January 2026)

Objective: Define project scope, objectives, and detailed planning through comprehensive documentation.

Key Activities:

1. **SMART Objectives Definition** (4h)
    - Specification of Main Objective (MO): Develop a reusable teaching package
    - Definition of 4 Specific Objectives (OE1-OE4) with measurable KPIs
    - Alignment with institutional GEISI curriculum requirements
2. **State of the Art Analysis** (15h)
    - Comprehensive review of 7 major cybersecurity training platforms
    - Analysis of existing laboratory platforms: HackTheBox, TryHackMe, Cisco Academy, SEED Labs, GOAD, VulnHub, Hack4u
    - Identification of pedagogical gaps and competitive advantages
    - Comparative analysis matrix across multiple dimensions
3. **Functional and Non-Functional Requirements** (10h)
    - RF1: Design and implement 4 EVE-NG topologies (.unl files)
    - RF2: Implement vulnerabilities per PTES**ptes**, OWASP**owasp_testing_guide**, NIST**nist_framework**, and CEH**ceh_framework** standards
    - RF3: Create structured assessment materials and rubrics

- NFR1: Performance optimization for deployment and execution
- NFR2: Compliance with international security frameworks
- NFR3: Institutional sustainability through open-source alignment

4. **Pentesting Methodology and Validation Framework** (8h)
   - Adaptation of PTES (Penetration Testing Execution Standard) for educational context
   - Definition of ethical hacking principles (EC-Council CEH framework)
   - Mapping of attack phases to lab objectives
   - Validation criteria for security effectiveness
5. **Design Decisions Justification** (5h)
   - Rationale for EVE-NG selection vs alternative platforms
   - Technology stack justification (Kali Linux, Metasploit, Burp, OWASP ZAP)
   - Pedagogical approach: hands-on labs with automation
6. **Risk Analysis and Mitigation Planning** (15h)
   - Identification of 12+ project risks (technical, pedagogical, institutional)
   - Assessment of impact and probability
   - Definition of mitigation strategies for each critical risk
7. **Resource Inventory and Allocation** (5h)
   - Hardware requirements: dual-socket server with Proxmox
   - Software stack: EVE-NG Professional, Kali Linux images, security tools
   - Human resources: student (320h), faculty supervisor, pilot users
   - Budget considerations and open-source alternatives
8. **Gantt Chart and Critical Path Analysis** (20h)

- Development of detailed project timeline with 37 tasks
- Identification of 13 critical path tasks
- CSV export and interactive web-based visualization
- Progress tracking and milestone definition

Deliverables:

- Formal project proposal (Avantprojecte/Preliminary Project document)
- Detailed Gantt chart with critical paths
- Risk register and mitigation plan
- Requirements specification document (functional and non-functional)

Duration: 82 hours | **Status:** In progress (70%)

Milestone Deadline: January 16, 2026

4.1.3 Phase 2: Implementation and Interim Validation (January-March 2026)

Objective: Develop the first three laboratories and conduct preliminary validation with pilot users.

Key Activities:

1. **Lab 1: Reconnaissance Development** (14h research + 20h dev)
    - Scenario: Intelligence gathering phase using PTES framework
    - Tools: Nmap**nmap**, Shodan, Passive reconnaissance techniques
    - Topology: Multi-tier network with passive monitoring capabilities
    - Validation: Successful execution of reconnaissance workflow

2. **Lab 2: Web Vulnerabilities Development** (14h research + 25h dev)
    - Scenario: OWASP Top 10 vulnerability exploitation
    - Tools: Burp Suite, OWASP ZAP, manual testing techniques
    - Topology: Web application server with vulnerable services
    - Validation: Exploitation success and reporting accuracy
3. **Lab 3: Network Analysis Development** (14h research + 25h dev)
    - Scenario: Network segmentation and lateral movement (NIST standards)
    - Tools: Wireshark**wireshark**, tcpdump, network enumeration tools
    - Topology: Multi-VLAN network with security monitoring
    - Validation: Successful network traversal and analysis
4. **Pilot Validation with Users** (20h + 6h revision)
    - Recruitment of 8-10 pilot users from GEISI cohort
    - Lab execution sessions with timing and metrics collection
    - User satisfaction surveys and usability assessment
    - Performance metrics: completion rates, average resolution time
5. **Topology Adjustments and Optimization** (30h)
    - Performance tuning based on pilot feedback
    - VM image optimization and deployment time reduction
    - Network configuration refinement for stability
6. **Technical Documentation** (40h)
    - Student manuals for each lab (15-20 pages minimum)
    - Teacher deployment guides and customization instructions
    - Assessment rubrics with quantifiable criteria

Deliverables:

- 3 fully functional and tested EVE-NG labs
- Student and teacher documentation
- Pilot validation report with metrics
- Technical write-ups and learning materials

Duration: 188 hours | **Status:** Planned

Milestone Deadline: March 20, 2026

4.1.4 Phase 3: Completion and Final Validation (March-May 2026)

Objective: Develop the fourth laboratory, complete automation, and conduct comprehensive final validation.

Key Activities:

1. **Lab 4: Privilege Escalation Development** (16h research + 30h dev)
    - Scenario: Post-exploitation and privilege escalation (CEH framework)
    - Tools: Metasploit, manual exploitation, system enumeration
    - Topology: Hardened systems with privilege escalation vectors
    - Validation: Successful privilege elevation and system compromise
2. **Comprehensive Penetration Testing** (30h Phase 1 + 25h Phase 2)
    - Phase 1: Sequential testing of Labs 1, 2, and 3
    - Phase 2: Integrated testing across all 4 laboratories
    - Documentation of all findings and remediation strategies
    - Validation of security effectiveness

3. **Automation Framework Development** (30h)
    - Deploy scripts (Bash/Python) for lab initialization
    - Reset scripts for state restoration
    - Validation scripts for automated verification
    - Integration into CI/CD pipeline
4. **Final User Validation and Iteration** (18h)
    - Extended pilot with 15-20 users over 2-week period
    - Collection of usage metrics and satisfaction data
    - Implementation of final improvements
    - User acceptance testing (UAT)
5. **Bachelor's thesis Report Writing and Finalization** (90h)
    - Bachelor's thesis Report Writing Phase I (15h): Introduction, Object, Literature Review
    - Bachelor's thesis Report Writing Phase II (40h): Methodology, Requirements, Implementation Results
    - Bachelor's thesis Report Writing Phase III (35h): Validation, Analysis, Conclusions
    - Final revision and quality assurance (15h)
    - Formatting, bibliography integration, and publication preparation (20h)

Deliverables:

- 4 fully functional and tested EVE-NG laboratories
- Complete automation framework (12+ scripts)
- Final bachelor's thesis report (70-120 pages)
- User validation report and lessons learned
- Deployment package ready for institutional use

Duration: 315 hours | **Status:** Planned

Final Milestone Deadline: May 27, 2026

4.2 Information Search Strategies

The research methodology employs multiple information gathering techniques:

4.2.1 Literature Review Approach

1. **Systematic Review of Academic Databases**
   - IEEE Xplore (computer science and engineering)
   - ACM Digital Library (cybersecurity education)
   - Scopus (multidisciplinary security research)
   - Google Scholar (open-access research)
2. **Industry Standards and Frameworks**
   - PTES (Penetration Testing Execution Standard) **ptes**
   - OWASP (Open Web Application Security Project) documentation **owasp_testing_guide**
   - NIST (National Institute of Standards and Technology) frameworks **nist_framework**, **nist_sp800115**
   - CIS Controls and benchmarks **cis_controls**
   - EC-Council CEH curriculum **ceh_framework**
3. **Platform and Tool Documentation**
   - Official EVE-NG documentation and community resources **eve_ng_docs**
   - Kali Linux penetration testing guide **kali_linux**
   - Metasploit Framework documentation **metasploit**
   - OWASP ZAP and Burp Suite user guides **owasp_zap**, **burp_suite**
4. **Comparative Analysis**
   - Feature comparison matrices for existing platforms
   - User reviews and community feedback
   - Educational effectiveness studies
   - Cost-benefit analysis of solutions

4.2.2 Primary Research Methods

1. **User Research and Validation**
   - Pilot testing with real students (10-20 participants)
   - Qualitative feedback through interviews
   - Quantitative metrics collection (time-on-task, completion rates)
   - Usability assessment using standardized scales
2. **Technical Experimentation**
   - Lab design iteration and refinement
   - Performance testing and optimization
   - Security effectiveness validation
   - Scalability assessment

4.3 Development Methodology

The project adopts an iterative, agile-inspired approach with clear phases and milestones:

4.3.1 Development Cycle

- **Design:** Conceptual architecture and topology planning
- **Implementation:** Lab creation, vulnerability implementation, tooling
- **Validation:** Security testing, educational effectiveness review
- **Iteration:** Feedback incorporation and refinement
- **Documentation:** Technical and pedagogical material preparation

4.3.2 Quality Assurance

1. **Technical Validation**
   - Automated testing of lab deployment and reset
   - Manual security testing and penetration verification
   - Performance benchmarking against KPI targets
   - Compatibility testing across EVE-NG versions
2. **Pedagogical Validation**
   - Alignment with GEISI learning outcomes
   - Assessment of student understanding through rubrics
   - Effectiveness in achieving educational objectives
   - Feedback from faculty and student participants
3. **Usability Testing**
   - User interface and documentation clarity
   - Accessibility and ease of deployment
   - Support and documentation adequacy
   - User satisfaction scoring

4.3.3 Documentation and Version Control

- GitHub repository for all code, scripts, and documentation
- LaTeX for formal project documentation (this bachelor's thesis report)
- Versioning system for all deliverables
- Change logs and improvement tracking
- Deployment guides and runbooks

4.4 Comparative Analysis

1. **HackTheBox**:
   - Strengths: Large collection of pre-built challenges, strong community
   - Limitations: Pre-built scenarios; limited network topology customization; external platform dependency; content aligned to commercial pentesting certifications, not specific degree curricula
   - Verdict: Unsuitable for curriculum-aligned, institution-controlled labs
2. **TryHackMe**:
   - Strengths: Gamified learning paths, beginner-friendly
   - Limitations: Content managed externally; platform-dependent; free tier is limited; scenarios often oversimplified for professional preparation
   - Verdict: Useful for introduction but insufficient for degree-level practical integration
3. **SEED Labs**:
   - Strengths: Comprehensive, research-backed cybersecurity exercises; Docker-based portability
   - Limitations: Fixed scenarios; limited virtualization topology control; minimal automation for lab reset
   - Verdict: Good for specific security concepts but less suitable for integrated network pentesting scenarios
4. **EVE-NG**:
   - Strengths: Full network topology customization; open-source; locally deployable; supports multiple virtualized OS; excellent automation capabilities; vendor-independent
   - Limitations: Steeper learning curve; requires institutional IT infrastructure
   - Verdict: Optimal for institution-controlled, curriculum-aligned, reusable labs

4.5 Key Advantages of EVE-NG

1. **Institutional Independence:** Open-source platform ensures no vendor lock-in, long-term sustainability, and institution-controlled customization
2. **Customization and Control:** Full topology design allows labs to reflect real infrastructure complexity (routing misconfigurations, network segmentation failures, hybrid cloud scenarios)
3. **Automation and Reproducibility:** Lab deployment, reset, and validation can be fully automated, enabling consistent student experiences and reducing instructor overhead
4. **Curriculum Integration:** Labs can embed specific technologies taught in parallel courses (OSPF routing, SQL databases, Active Directory authentication), creating the nexus where theoretical knowledge meets security contexts
5. **Cost-Effectiveness:** Zero licensing costs; compatible with open-source tools (Kali Linux, Metasploit Framework, Nmap); appropriate for educational institutions with limited budgets
6. **Scalability:** Lab files are portable and exportable, enabling future deployment across multiple course sections, institutions, or as a resource for the broader educational community

4.6 Validation and Success Criteria

Success measurement is based on the KPIs defined in Chapter 3 (Objectives and Scope), organized into three categories:

4.6.1 Technical Success Criteria

- Lab deployment time $\leq$ 2 minutes per lab (95% of cases)
- VM boot success rate $\geq$ 95%
- Full reset time $\leq$ 30 seconds

- Scenario reproducibility: 100% repeatability
- Zero critical incidents in production environment

4.6.2 Educational Success Criteria

- Average lab resolution time: 90-120 minutes
- Successful completion rate $\geq$ 85% (pilot with 10+ users)
- Key concept understanding $\geq$ 80% correct
- User satisfaction score $\geq$ 4/5
- Assessment alignment with GEISI curriculum

4.6.3 Sustainability Success Criteria

- Full documentation coverage (100% of features)
- Open-source tools only (ensuring long-term maintainability)
- Compatibility with EVE-NG Community and Professional editions
- Institutional readiness for multi-year deployment
- Faculty capacity to deploy and customize

4.7 Risk Management

Risk mitigation is embedded throughout the project phases:

4.7.1 Key Risks and Mitigation

Risk	Mitigation Strategy	Owner
Scope creep / Feature additions	Clear scope boundaries and change management	Eloi
EVE-NG compatibility issues	Extensive testing on both Community and Pro editions	Eloi
Pilot user dropout	Clear incentives and scheduling flexibility	Pere + Eloi
Performance degradation	Early benchmarking and optimization iterations	Eloi
Documentation gaps	Peer review and user feedback cycles	Eloi + Faculty
Institutional adoption barriers	Change management plan and training support	Pere

Table 4.1: Key project risks and mitigation strategies

5 Functional and Non-Functional Requirements

This chapter specifies the complete requirements for the EVE-NG-based **eve_ng** cybersecurity teaching package, encompassing functional requirements (RF) that define what the system must do, technological requirements (TR) that specify the technical environment, and non-functional requirements (NFR) that address quality attributes and compliance standards.

5.1 Functional Requirements

Functional requirements define the core capabilities and behaviors expected from the teaching package.

5.1.1 RF1: Design and Implement EVE-NG Topologies

Description: Create four separate, functional, and interconnected network topologies in EVE-NG format (.unl files) that simulate real-world cybersecurity scenarios.

Detailed Requirements:

- **RF1.1**: Design 4 distinct network topologies, each targeting a specific pentesting phase:
  - Lab 1: Reconnaissance topology (passive intelligence gathering)
  - Lab 2: Web application security topology (active vulnerability discovery)
  - Lab 3: Network analysis topology (lateral movement and monitoring)
  - Lab 4: Privilege escalation topology (post-exploitation scenarios)
- **RF1.2**: Each topology must include:

  - Minimum 5-8 virtual machines per lab
  - Realistic network architecture (DMZ, internal networks, monitoring zones)
  - Vulnerable services configured per OWASP and NIST standards
  - Network segmentation with VLANs where appropriate
- **RF1.3**: EVE-NG compatibility requirements:
  - .unl file format compatible with EVE-NG Community Edition
  - Additional compatibility with EVE-NG Professional Edition
  - KVM hypervisor support (Ubuntu 20.04+ LTS)
  - Minimum host specification: 16GB RAM, 200GB storage
- **RF1.4**: Topology documentation:
  - Network diagram for each lab (logical and physical views)
  - IP addressing scheme documented
  - Service inventory and port mappings
  - Attack surface identification and justification

Success Criteria:

- All 4 .unl files deploy successfully without errors
- VM boot success rate greater or equal to 95 percent
- Network connectivity verified between all nodes
- Estimated deployment time: less or equal to 2 minutes per lab

5.1.2 RF2: Implement Vulnerabilities and Attack Scenarios

Description: Implement realistic vulnerabilities and attack scenarios aligned with industry frameworks (PTES**ptes**, OWASP**owasp_testing_guide**, NIST**nist_framework**, CEH**ceh_framework**).

Detailed Requirements:

- **RF2.1**: Lab 1 - Reconnaissance (PTES Phase 1)
  - Passive reconnaissance tools (Shodan, DNS enumeration)
  - Active scanning tools (Nmap**nmap**, banner grabbing)
  - Service discovery scenarios
  - Information gathering reporting
- **RF2.2**: Lab 2 - Web Vulnerabilities (OWASP Top 10)
  - Injection vulnerabilities (SQL, command injection)
  - Authentication bypasses
  - Session management flaws
  - Cross-Site Scripting (XSS) scenarios
  - Cross-Site Request Forgery (CSRF) vulnerabilities
  - Broken Access Control
  - Security misconfiguration examples
  - Sensitive data exposure scenarios
- **RF2.3**: Lab 3 - Network Analysis (NIST Standards)
  - Network segmentation analysis
  - Lateral movement techniques
  - Man-in-the-Middle (MITM) attack scenarios
  - Network traffic analysis with Wireshark**wireshark**
  - IDS/IPS evasion scenarios
  - VPN and encryption bypass (where applicable)
- **RF2.4**: Lab 4 - Privilege Escalation (CEH Framework)
  - Local privilege escalation vulnerabilities
  - Kernel exploits (outdated systems)
  - Weak file permissions and sudo misconfigurations
  - Windows privilege escalation (UAC bypass, token impersonation)
  - Post-exploitation persistence mechanisms
- **RF2.5**: Vulnerability implementation standards:

  - All vulnerabilities must be fixable by administrators
  - No permanently broken or unrecoverable systems
  - Clear reset procedure for each vulnerability state
  - Automated scanning tools must successfully identify vulnerabilities

Success Criteria:

- Minimum 3-4 vulnerability scenarios per lab
- 100 percent of implemented vulnerabilities exploitable through documented methods
- Average exploitation time: 15-45 minutes per vulnerability
- All vulnerabilities remediable with documented procedures

5.1.3 RF3: Create Structured Assessment Materials

Description: Develop comprehensive assessment rubrics, learning objectives, and evaluation materials aligned with GEISI curriculum **geisi_curriculum.**

Detailed Requirements:

- **RF3.1:** Learning outcomes definition:
  - 3-4 specific learning outcomes per lab
  - Alignment with BLOOM's taxonomy (knowledge, application, analysis)
  - Clear connection to GEISI degree competencies
- **RF3.2:** Assessment rubrics:
  - Detailed rubrics for each lab (minimum 15 pages per lab)
  - Quantifiable evaluation criteria
  - Point allocation per criterion (clear grading scale)
  - Example student responses (exemplars)
- **RF3.3:** Support materials:

  - Student lab manuals with step-by-step guidance
  - Teacher guides with deployment and customization instructions
  - Expected outcomes and common pitfalls documentation
  - Troubleshooting guides for common technical issues
- **RF3.4**: Validation materials:
  - Automated validation scripts to verify successful lab completion
  - Manual verification procedures for educators
  - Documentation of success indicators

Success Criteria:

- 4 complete assessment rubrics (1 per lab)
- 4 student manuals (minimum 15 pages each)
- 1 comprehensive teacher guide
- 100 percent of features documented with examples

5.2 Technological Requirements

Technological requirements specify the infrastructure, tools, and platforms required for system implementation and deployment.

5.2.1 TR1: Virtualization Infrastructure

- **TR1.1**: EVE-NG Platform
  - Version: EVE-NG 5.0 or later
  - Editions supported: Community (free) and Professional
  - Deployment: Docker container or bare-metal installation
  - Web interface: Accessible via HTTPS on port 443
  - API: RESTful API for automation and integration

- **TR1.2**: Hypervisor
  - Primary: KVM (Kernel-based Virtual Machine)
  - Host OS: Ubuntu Server 20.04 LTS or 22.04 LTS
  - QEMU integration for VM management
  - Nested virtualization support for advanced scenarios
- **TR1.3**: Host System Specifications
  - CPU: 2x Intel Xeon E5-2600 v3 series or equivalent (minimum 16 cores)
  - RAM: 64GB minimum (recommended 128GB for simultaneous labs)
  - Storage: 2TB+ NVMe or SSD (minimum 50GB per VM image)
  - Network: Gigabit Ethernet (10Gbps preferred)
  - Power redundancy: UPS for service continuity
- **TR1.4**: Network Configuration
  - Layer 2 switching capabilities within EVE-NG
  - VirtualBox-based network simulation where needed
  - VLAN support for network segmentation scenarios
  - IPv4 and IPv6 support

5.2.2 TR2: Security Tools and Penetration Testing Framework

- **TR2.1**: Attacking Platform
  - Primary: Kali Linux 2024.x **kali_linux**
  - Alternative: Parrot Security OS **parrot_os**
  - Installation method: EVE-NG pre-built image or custom import
  - Tools package: Full toolkit including reconnaissance, scanning, and exploitation
- **TR2.2**: Core Security Tools (installed on Kali)

  - **Reconnaissance:** Nmap, Shodan CLI, Amass, Sublist3r
  - **Scanning:** Nessus (optional), OpenVAS, Nikto
  - **Exploitation:** Metasploit Framework 6.x **metasploit**, ExploitDB, Searchsploit
  - **Web Testing:** Burp Suite Community/Professional **burp_suite**, OWASP ZAP **owasp_zap**
  - **Network Analysis:** Wireshark**wireshark**, tcpdump, mitmproxy
  - **Post-Exploitation:** Mimikatz, Rubeus, BloodHound
  - **Programming:** Python 3.10+, Bash scripting

- **TR2.3:** Target System Images
  - Ubuntu Server 18.04/20.04 LTS (Linux targets)
  - CentOS 7/8 (enterprise Linux)
  - Windows Server 2016/2019 (Windows targets)
  - Vulnerable applications: DVWA, WebGoat, Juice Shop, Mutillidae

- **TR2.4:** Compliance and Documentation
  - All tools must be freely available or have educational licenses
  - Open-source preference for long-term sustainability
  - Tool version specifications documented
  - Legal and ethical use guidelines included

5.2.3 TR3: Scripting and Automation Framework

- **TR3.1:** Languages
  - Bash scripting for system automation (deployment, reset, validation)
  - Python 3.10+ for complex automation and integration
  - Optional: Go or Rust for performance-critical components

- **TR3.2:** Automation Scripts

  - Deploy scripts: Initialize lab topologies and configurations
  - Reset scripts: Restore lab to initial vulnerable state
  - Validation scripts: Verify security effectiveness and readiness
  - Monitoring scripts: Track lab health and performance
- **TR3.3**: Integration Points
  - EVE-NG API integration for programmatic lab management
  - Webhook support for event-driven automation
  - Container orchestration (Docker) for microservices
  - Version control integration (Git, GitHub)

5.3 Non-Functional Requirements

Non-functional requirements specify quality attributes and operational characteristics.

5.3.1 NFR1: Performance and Efficiency

- **NFR1.1**: Deployment Performance
  - Lab deployment time: less or equal to 2 minutes from topology load to ready state
  - VM boot time: less or equal to 90 seconds per VM (95 percent of cases)
  - Reset operation: less or equal to 30 seconds to restore to initial state
  - Concurrent lab instances: Support minimum 5 simultaneous labs
- **NFR1.2**: Resource Optimization
  - VM image sizes: less or equal to 5GB per image (compressed)
  - Memory footprint: less or equal to 6GB per lab instance during execution

  - Network bandwidth: Minimal external dependencies (all local simulation)

- **NFR1.3**: Scalability
  - Support for 20+ simultaneous students (1 lab per student minimum)
  - Support for 30+ concurrent educational deployments
  - Horizontal scaling through distributed EVE-NG nodes

5.3.2 NFR2: Compliance with Security Frameworks

- **NFR2.1**: NIST Cybersecurity Framework Alignment **nist_framework**
  - Core Functions: Identify, Protect, Detect, Respond, Recover
  - Implementation mapped to lab exercises
  - Assessment procedures aligned with NIST guidelines
- **NFR2.2**: CIS Critical Controls Coverage **cis_controls**
  - Controls 1-18 incorporated where pedagogically appropriate
  - Assessment rubrics aligned with CIS benchmarks
  - Remediation procedures follow CIS recommendations
- **NFR2.3**: OWASP Standards Integration **owasp2021**, **owasp_testing_guide**
  - OWASP Top 10 vulnerabilities fully covered
  - OWASP Testing Guide methodology integrated
  - OWASP Development Practices reflected in lab design
- **NFR2.4**: Penetration Testing Standards **ptes**, **ceh_framework**
  - PTES (Penetration Testing Execution Standard) phases mapped to labs
  - CEH (Certified Ethical Hacker) curriculum alignment
  - Ethical hacking principles strictly enforced

5.3.3 NFR3: Reliability and Availability

- **NFR3.1**: System Reliability
  - Mean Time Between Failures (MTBF): greater or equal to 100 hours continuous operation
  - Mean Time To Recovery (MTTR): less or equal to 5 minutes
  - Scenario reproducibility: 100 percent (deterministic behavior)
  - Bug severity: Zero critical bugs at release
- **NFR3.2**: Data Integrity
  - No unintended data loss during lab operations
  - Snapshot and restore functionality for state management
  - Backup procedures for sensitive configurations
- **NFR3.3**: Availability
  - Planned maintenance windows: Weekends only
  - System availability: greater or equal to 99 percent during instructional periods
  - Graceful degradation if resources become limited

5.3.4 NFR4: Security and Ethical Compliance

- **NFR4.1**: Educational Sandbox Isolation
  - Complete network isolation from production systems
  - No outbound internet access from lab environments
  - Contained attack scenarios (no external targets)
- **NFR4.2**: Access Control and Accountability
  - Role-based access control (RBAC) for lab management
  - Audit logs of all student actions (where applicable)
  - Clear identification of authorized vs. unauthorized activities

- **NFR4.3**: Ethical Guardrails
  - Mandatory ethics training and certification before lab access
  - Written agreements regarding responsible use
  - Monitoring and escalation procedures for suspicious activities

5.3.5 NFR5: Usability and Support

- **NFR5.1**: Ease of Use
  - Average time to deploy a lab: 5-10 minutes for faculty
  - Intuitive web interface with clear navigation
  - Context-sensitive help and tooltips
  - Minimal technical prerequisites for students
- **NFR5.2**: Documentation and Support
  - Comprehensive user manuals and quick-start guides
  - Video tutorials for complex procedures (2-3 minutes each)
  - FAQ documentation (minimum 50 entries)
  - Responsive support channel (email, discussion forum, chat)
- **NFR5.3**: Accessibility
  - WCAG 2.1 Level AA compliance for web interfaces
  - Screen reader compatibility
  - Keyboard navigation support
  - Clear contrast ratios for visual elements

5.3.6 NFR6: Institutional Sustainability

- **NFR6.1**: Long-term Maintainability
  - All components open-source or freely available tools
  - No vendor lock-in dependencies

  - Source code in GitHub with clear documentation
  - Backward compatibility with older EVE-NG versions where feasible

- **NFR6.2**: Curriculum Alignment
  - Explicit mapping to GEISI degree learning outcomes
  - Integration with existing course structures
  - Flexible customization for different instructional contexts
  - Support for multi-level deployment (100/300/400-level courses)

- **NFR6.3**: Scalability for Institutional Adoption
  - Deployable on institutional infrastructure (not SaaS-dependent)
  - Support for multiple faculty deployments simultaneously
  - Resource reservation and scheduling capabilities
  - Reporting and analytics for institutional assessment

5.4 Requirements Traceability Matrix

ID	Type	Requirement	Scope
RF1.1	Functional	Design 4 EVE-NG topologies	All labs
RF1.2	Functional	5–8 VMs per topology	All labs
RF2.1	Functional	Lab 1 reconnaissance scenarios	Lab 1
RF2.2	Functional	Lab 2 OWASP vulnerabilities	Lab 2
RF2.3	Functional	Lab 3 network analysis	Lab 3
RF2.4	Functional	Lab 4 privilege escalation	Lab 4
RF3.1	Functional	Learning outcomes defined	All labs
RF3.2	Functional	Assessment rubrics	All labs
TR1.1	Technical	EVE-NG 5.0+	Platform
TR1.3	Technical	Host 64GB RAM, 2TB storage	Infrastructure
TR2.1	Technical	Kali Linux 2024 as attacking platform	Tools
NFR1.1	Non-Func.	Deploy in $\leq$ 2 minutes	Performance
NFR2.1	Non-Func.	NIST alignment	Compliance
NFR3.1	Non-Func.	MTBF $\geq$ 100h	Reliability
NFR4.1	Non-Func.	Network isolation	Security
NFR6.1	Non-Func.	Open-source tools	Sustainability

Table 5.1: Requirements traceability matrix for the EVE-NG laboratory platform

6 Development

This chapter describes the implementation of the penetration testing lab environment built on EVE-NG. It covers the platform setup, the management of virtual machine images, the lab management interface, the infrastructure problems encountered during development, and an overview of the lab topologies designed for the course. Each individual lab is then described in its own section.

6.1 Platform Setup

The development process began with the installation and validation of EVE-NG **eve_ng** as the network emulation platform. Following the official EVE-NG documentation, an initial environment was deployed in order to verify basic functionality: creating simple network topologies, deploying virtual machines with different operating systems, and testing console and remote desktop access before committing to the full lab design.

Several attempts were made to run EVE-NG as a virtual machine on a VMware-based infrastructure available in the initial university environment. Despite VMware being an officially supported hypervisor, recurrent stability and integration issues prevented a reliable setup. After evaluating the available alternatives and in agreement with the project supervisor, the platform was migrated to a dedicated Proxmox VE **proxmox** server. This choice offers better compatibility with the nested virtualisation required by EVE-NG and can be replicated on any institutional server running the same hypervisor.

On top of Proxmox, an EVE-NG virtual machine was deployed following the official installation guidelines, with the networking model adapted to the Proxmox bridge configuration. Console access via Telnet and graphical access via VNC were validated successfully for all node types used in the labs. Native Wireshark integration was not enabled in this iteration due to conflicts with pre-existing

software on the host machine, and was therefore left out of scope for the current version.

6.2 Image Management

6.2.1 Image Selection

Once the platform was stable, a survey of available EVE-NG-compatible images was conducted, prioritising free and open-source operating systems suitable for an academic context. Starting from a broad list of candidate images, the final selection was reduced to a minimal, reusable core for the penetration testing scenarios:

- **VyOS vyos**: used as a router and Layer 3 switch for routing and inter-VLAN scenarios.
- **pfSense CE pfsense**: used as the perimeter firewall, providing NAT, DNS resolution, and port forwarding.
- **GNU/Linux distributions** (Ubuntu Server 24.04 **ubuntu**, Ubuntu Desktop 24.04, Parrot OS **parrot_os**): used as target servers, internal workstations, and attacker machines respectively.

This reduced set balances realism and manageability, keeps all software licences aligned with the academic context, and limits the overhead of maintaining multiple image versions across labs.

6.2.2 Image Organisation in EVE-NG

To ensure consistency and reproducibility, all images were organised under the standard EVE-NG path `/opt/unetlab/addons/qemu/`. Each system was placed in a dedicated directory with a normalised name following the pattern `<type>-<version>` (e.g., `pfsense-2.7.2`, `vyos-1.x-rolling-amd64`, `linux-ubuntu-server-24.04`).

For GNU/Linux systems, the `linux-` prefix was applied consistently (e.g., `linux-ubuntu-desktop-24.04`, `linux-parrot-rolling`), since EVE-NG only makes the *Linux* node category available in

the interface when at least one directory with this prefix exists. Each directory contains a bootable image named `cdrom.iso` and a `virtioa.qcow2` disk image sized according to the role of the node.

6.2.3 ISO Upload Automation

The manual workflow for preparing each image — creating the directory, uploading the ISO, renaming it, creating the disk, and fixing permissions — proved repetitive and error-prone during initial experiments. To address this, a Python script named `iso_uploader.py` was developed as part of the project tooling.

The script presents a graphical file picker, uploads the selected ISO files to the EVE-NG server via SCP, and then automates the remaining preparation steps: directory creation under `/opt/unetlab/addons/qemu`, renaming the ISO to `cdrom.iso`, creation of the `virtioa.qcow2` disk at a size inferred from the image type, and execution of `unl_wrapper -a fixpermissions`. This tool reduces the time required to onboard a new image and enforces a uniform directory structure across deployments.

6.3 EVE-NG Lab Management Interface

EVE-NG provides a web-based graphical interface through which administrators and students can manage lab files independently of each other. The *File Manager*, accessible after login, is the main entry point for lab operations and offers several features that are relevant to reproducibility and multi-user deployments:

- **Lab import and export:** Topology files (`.unl`) can be imported by dragging them into the interface. Exported archives preserve the topology, node configurations, and snapshots, making it straightforward to share or back up a complete lab.
- **Folder structure:** Labs can be organised hierarchically (e.g., `/GEISI-Labs/Pentesting/`), which keeps different courses

and iterations separate without requiring multiple EVE-NG installations.

- **Multi-lab support**: Several topologies can run simultaneously. The administrator dashboard provides a real-time view of CPU and memory consumption per lab, which is useful for capacity planning on shared infrastructure.

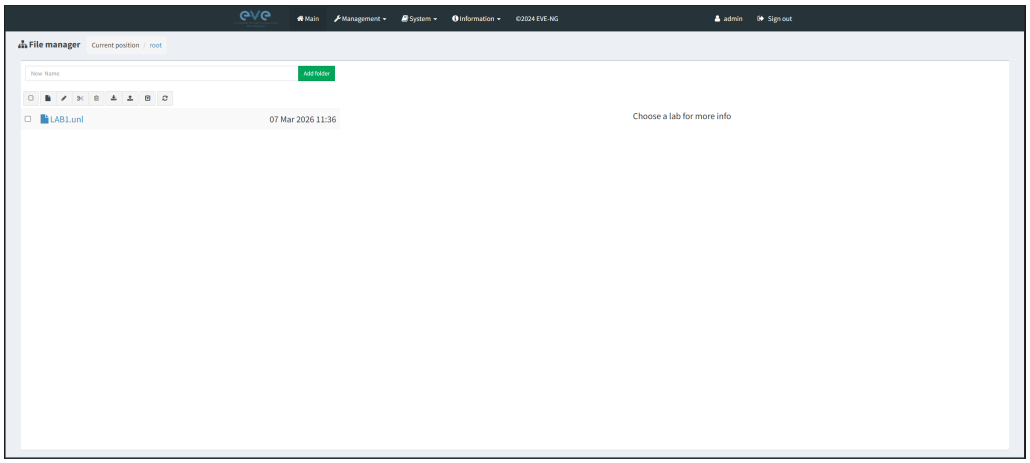

Figure 6.1: EVE-NG File Manager showing the lab directory structure, import controls, and topology thumbnails.

The export format also simplifies distribution: a `.unl` file committed to the project repository can be imported into any compatible EVE-NG instance, reproducing the full topology without manual reconfiguration.

6.4 Infrastructure Problems

Several problems were encountered at the infrastructure level during the setup of the EVE-NG environment. These are distinct from the lab-specific issues documented within each lab section and relate to the underlying platform and tooling.

6.4.1 EVE-NG on VMware

The initial deployment target was a VMware-based server available through the university. Despite EVE-NG being officially listed

as compatible with VMware **vmware_esxi** hypervisors, the environment produced recurrent instability: nodes failed to start consistently, VNC sessions dropped without warning, and network connectivity between virtual bridges was unreliable. No stable configuration was found after several iterations. The root cause was not fully identified, but the combination of VMware's nested virtualisation restrictions and the specific hardware and software versions in the university environment appeared to be contributing factors. The decision to migrate to Proxmox resolved all of these issues.

6.4.2 Node Configuration Persistence

A recurring challenge in EVE-NG lab design is ensuring that node configurations survive restarts. By default, runtime network settings applied inside guest virtual machines are lost when the lab is stopped or the EVE-NG host is rebooted. This affected multiple nodes across the lab and required a dedicated solution for each one. The specific mechanisms used are described in the relevant lab sections; the general principle adopted throughout the project was to rely on the native persistence mechanisms of each operating system — such as `systemd-networkd` unit files, NetworkManager connection profiles, and `udev` rules — rather than applying settings interactively at the console.

6.4.3 LVM Volume Group Naming Conflict

During a session involving disk operations on guest virtual machine images mounted from the EVE-NG host, a naming conflict arose between the host's LVM volume group and a guest disk's volume group, both named `ubuntu-vg`. An attempt to resolve the conflict by renaming the host volume group caused the EVE-NG host to fail on the next restart, dropping into an `initramfs` prompt because the boot configuration still referenced the original group name.

Recovery was performed from the `initramfs` shell by renaming the volume group back to its original name and reactivating it before resuming the normal boot sequence. As a result of this incident, the

rule established for the project is to never rename a volume group belonging to the host operating system. When a naming conflict arises with a guest disk, only the guest volume group is renamed, using a descriptive identifier that includes the lab and node reference.

6.5 Lab Topologies

Four penetration testing lab topologies were designed for the course. Each topology is implemented as an EVE-NG `.unl` file and can be imported into any compatible EVE-NG instance. Table ?? summarises the four labs, their primary learning objective, and their current implementation status.

Lab	Primary Objective
Lab 1: Reconnaissance & Enumeration	Port scanning, web content discovery, SSH
Lab 2: Web Application Vulnerabilities	OWASP Top 10 exploitation **owasp2021**
Lab 3: Network Analysis	Traffic capture, lateral movement
Lab 4: Privilege Escalation	Local privilege escalation techniques

Table 6.1: Overview of the four EVE-NG lab topologies designed for the course.

The remainder of this chapter describes each implemented lab in detail, covering the network topology, design decisions, student workflow, and problems encountered during development.

6.6 Lab 1: Reconnaissance and Web Enumeration

6.6.1 Overview and Learning Objectives

The first laboratory focuses on the initial phases of a penetration test: passive and active reconnaissance, service enumeration, and web content discovery. Rather than presenting vulnerabilities in isolation, the lab situates them within a realistic multi-layer network architecture that the student must first understand before being able to progress.

The intended learning outcomes are:

- Perform DNS-based hostname resolution and understand how name services relate to firewall and routing configuration.
- Identify services exposed through a perimeter firewall using port scanning and HTTP enumeration techniques.
- Discover sensitive web content through source code analysis and directory enumeration.
- Establish an authenticated SSH session using credentials obtained through reconnaissance, and locate flag artefacts on the target system.

The lab is designed to be completed sequentially: each step reveals the information needed for the next, reinforcing the idea that reconnaissance is a structured, iterative process rather than a single action.

6.6.2 Network Topology

The topology consists of five nodes interconnected across three network segments, as illustrated in Figure ??.

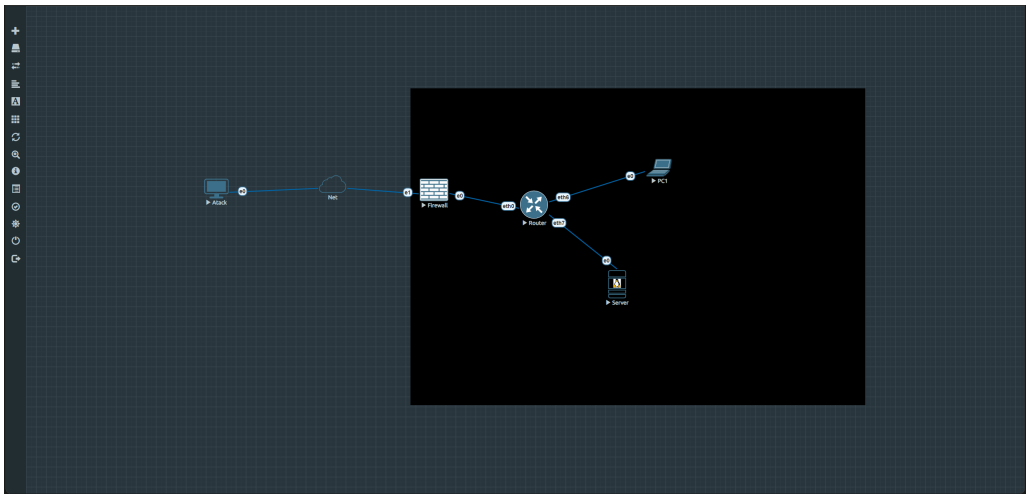

Figure 6.2: Lab 1 network topology: attacker node, perimeter firewall, internal router, target server and workstation.

The **Attacker** node (Parrot OS **parrot_os**) is connected to the simulated external network, which mirrors a standard institutional LAN.

From this position, the attacker has no direct visibility of the internal segments and must work through the firewall.

The **pfSense pfsense** firewall connects the external segment to an internal transit link. It handles perimeter NAT, DNS resolution for the `lab1` hostname, and port forwarding rules that expose specific services on the target server to the outside.

The **VyOS vyos** router sits behind pfSense and performs Layer 3 routing between two internal segments: a servers network and a users network. This separation reflects a realistic campus or enterprise design where different host categories are isolated at the network layer.

The **Server** node (Ubuntu Server 24.04 **ubuntu**, hostname `pebcak`) runs an nginx **nginx** web server and is the primary target of the lab. The **PC1** node (Ubuntu Desktop 24.04) represents an internal workstation and is reserved for future lab extensions.

Network Addressing

Table **??** summarises the IP addressing scheme used across all nodes and interfaces.

Table 6.2: Lab 1 — IP addressing table

Node	Interface	IP Address	Role
pfSense	vtnet1 (WAN)	192.168.0.29/24	Student entry point
pfSense	vtnet0 (LAN)	172.16.0.1/30	Transit link to VyOS
VyOS	eth0	172.16.0.2/30	Link to pfSense
VyOS	eth6	192.168.10.5/24	Users LAN gateway
VyOS	eth7	192.168.20.1/24	Servers LAN gateway
Server (pebcak)	ens3	192.168.20.50/24	Target — web + SSH
PC1	ens3	192.168.10.50/24	Internal workstation
Attacker (Parrot)	pnet0	DHCP / 192.168.0.x	Pentesting node

6.6.3 Infrastructure Design Decisions

Several design decisions were made to balance realism, educational value, and maintainability.

Layered Network Architecture

A flat network with a single target machine would be simpler to configure, but would fail to represent the kind of environment a student is likely to encounter in a real engagement. The three-segment design — external, transit, and internal — forces the student to reason about routing and NAT before attempting any exploitation, which aligns with the reconnaissance-first philosophy of the lab.

DNS-Based Access

The target is intentionally not accessible by raw IP address in the lab instructions. Instead, the pfSense DNS Resolver **pfsense** is configured to resolve a custom hostname (`lab1`) to the firewall's WAN address. This introduces students to the role of DNS in reconnaissance and reflects real-world scenarios where targets are identified by name rather than by address.

The Unbound **unbound_dns** custom options stored in pfSense are:

```
interface: 192.168.0.29
access-control: 192.168.0.0/24 allow
local-zone: "lab1." redirect
local-data: "lab1. A 192.168.0.29"
```

Listing 6.1: pfSense Unbound custom options for lab1 hostname

NAT Port Forwarding

The pfSense firewall exposes three services from the internal server to the external segment via static port-forward rules, as shown in Table **??**.

Table 6.3: Lab 1 — pfSense NAT port-forward rules (WAN → Server)

Protocol	WAN port	Target	Service
TCP	80	192.168.20.50:80	HTTP — nginx web server
TCP	443	192.168.20.50:443	HTTPS — nginx web server
TCP	22	192.168.20.50:22	SSH — target system access

CTF-Style Progression

The lab uses a Capture The Flag (CTF) model with a linear chain of discoveries. Each finding unlocks the next step, which mirrors the iterative nature of real penetration tests while providing clear checkpoints for assessment. The chain is described in detail in Section **??**.

6.6.4 Target Configuration — PEBCAK Corp

The target server hosts a fictional company portal under the name *PEBCAK Corp* (an acronym for *Problem Exists Between Chair And Keyboard*, a common expression in IT support). The theme is intentionally self-referential: a company whose name implies human error is itself misconfigured.

The web application consists of two pages served by nginx:

- **Index page** (`/`): A corporate landing page with minimal content. An HTML comment embedded in the source code provides a subtle hint directing the student towards further enumeration.
- **Internal page** (`/pebcak.html`): A page styled as an internal maintenance document, exposing SSH credentials for the server. It is not linked from the index and must be discovered independently.

Once SSH access to the server is established, a plain-text flag file is present in the user's home directory. The file also contains credentials for the internal firewall management interface, providing a pivot point for students who wish to explore beyond the primary objective.

6.6.5 Student Workflow

The intended progression through the lab is as follows.

Step 1 — DNS Resolution

The student configures the attacker node to use the pfSense Unbound resolver and verifies that the `lab1` hostname resolves correctly:

```
$ dig lab1 @192.168.0.29

;; ANSWER SECTION:
lab1.                    0       IN      A       192.168.0.29

$ resolvectl query lab1
lab1: 192.168.0.29
```

Listing 6.2: DNS resolution of the lab1 hostname

Step 2 — Service Enumeration

A version and script scan against the resolved address reveals the exposed services:

```
$ nmap -sV -sC -p 22,80,443 lab1

PORT    STATE SERVICE  VERSION
22/tcp  open  ssh      OpenSSH 9.6p1 Ubuntu 4ubuntu0.4
80/tcp  open  http     nginx 1.24.0 (Ubuntu)
| http-title: PEBCAK Corp -- Internal Portal
443/tcp closed https

Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Listing 6.3: nmap service enumeration of the lab1 target

The results confirm HTTP and SSH are reachable through the firewall. HTTPS is closed, consistent with the nginx configuration on the server.

Step 3 — Web Content Discovery

Browsing to `http://lab1` presents the PEBCAK Corp index page. Inspecting the HTML source reveals a comment hinting at a hidden path:

```
$ curl -s http://lab1 | grep -i "comment\|<!--"

<!-- TODO: remove /pebcak.html before going public - IT dept -->
```

Listing 6.4: HTML source inspection of the index page

Step 4 — Credential Retrieval

Navigating to the discovered path exposes SSH credentials:

```
$ curl -s http://lab1/pebcak.html

<html>
<body>
  <h1>PEBCAK Corp -- Internal Maintenance</h1>
  <p>SSH access for sysadmin team:</p>
  <p>User: <b>pebcak</b> | Pass: <b>pebcak123</b></p>
  <p>Host: lab1 (port 22)</p>
</body>
</html>
```

Listing 6.5: Retrieving credentials from the hidden page

Step 5 — SSH Access and Flag Capture

Using the retrieved credentials, the student establishes an SSH session and locates the flag:

```
$ ssh pebcak@lab1

pebcak@pebcak:~$ ls
flag.txt

pebcak@pebcak:~$ cat flag.txt
```

```
FLAG{p3bc4k_s3rv3r_0wn3d}

pfSense admin credentials (bonus): admin / pfsense
```

Listing 6.6: SSH login and flag retrieval

Optional Extension — Firewall Pivot

Students who wish to explore further can use the pfSense credentials found in the flag file to access the firewall management interface:

```
$ ssh admin@172.16.0.1

[2.7.2-RELEASE][admin@pfSense.home.arpa]/root:
```

Listing 6.7: Accessing pfSense management via SSH (optional)

This demonstrates the cascading impact of credential exposure: a single misconfigured web page leads to full administrative access over the network perimeter.

6.6.6 Persistent Environment Configuration

The following mechanisms were implemented to ensure that all Lab 1 node configurations survive a restart of the lab or the EVE-NG host:

- **Server network configuration:** Managed via `systemd-networkd` with a static unit file, ensuring the server's IP address and default gateway are restored automatically on boot.
- **Bridge interface addresses:** The EVE-NG host requires IP addresses on its internal bridge interfaces to route management traffic to the lab nodes. These are applied at boot using a `udev` rule that triggers on interface creation.
- **Attacker DNS configuration:** The attacker node's DNS resolver is set to the pfSense WAN address via NetworkManager, with

DHCP-provided DNS overridden. This setting is stored in the NetworkManager connection profile and persists across reboots.

- **pfSense DNS Resolver:** Custom Unbound **unbound_dns** options are stored in the pfSense configuration file (`config.xml`) and applied at startup by the pfSense boot sequence.

6.6.7 Problems Encountered and Solutions

VNC Keyboard Input on QEMU Nodes

Early attempts to automate node configuration through VNC used the `vncdo` command-line utility. While screenshot capture worked correctly, key event injection consistently caused the connection to drop on QEMU-based nodes. Investigation revealed that pfSense's QEMU VNC implementation closes the session after receiving certain KeyEvent messages sent by `vncdo`.

The solution was to implement VNC input directly over a raw TCP socket using the RFB protocol. Key events are constructed manually as binary structs and sent without relying on any third-party VNC client library. This approach proved stable across all QEMU nodes in the lab.

pfSense DNS Resolver WAN Interface

Configuring pfSense's Unbound DNS Resolver to respond to queries arriving on the WAN interface required two separate changes: adding a listen directive for the WAN address and adding a corresponding access control entry permitting queries from the external subnet. The pfSense configuration system stores these options as a Base64-encoded string inside `config.xml`. Providing the options as plain text caused them to be decoded as binary garbage when the configuration was applied, resulting in a silently broken Unbound configuration. Once the encoding requirement was identified, the options were stored correctly and the resolver responded as expected.

Unbound Startup Configuration Path

After restarting the Unbound service using the standard FreeBSD service command, the process started with the default configuration file rather than the one generated by pfSense. This caused the DNS resolver to listen only on the loopback interface, ignoring the custom options. The root cause was that the FreeBSD service script references `/usr/local/etc/unbound/unbound.conf` by default, while pfSense generates its runtime configuration at `/var/unbound/unbound.conf`. The correct approach is to trigger a restart through pfSense's own PHP service management functions, which regenerate the configuration file before starting the process.

VyOS Source NAT Requirement

Initial tests showed that HTTP responses from the server were reaching the attacker but connection resets occurred intermittently. The root cause was asymmetric routing: reply packets from the server (192.168.20.50) were routed through VyOS back to pfSense with source address 192.168.20.50, but pfSense's state table expected replies to arrive on the LAN interface from the transit subnet. Adding a Source NAT rule on VyOS to masquerade traffic leaving the external-facing interface resolved the asymmetry and stabilised all TCP connections.

6.6.8 Results and Conclusions

Flags Obtained

Table 6.4: Lab 1 — captured flags

Flag	Value	Location
Primary flag	`FLAG{p3bc4k_s3rv3r_0wn3d}`	`~/flag.txt` on server
Bonus access	pfSense admin credentials	Embedded in flag file

Vulnerabilities Demonstrated

The lab successfully demonstrates several categories of security weakness that appear regularly in real-world environments:

- **Information disclosure via HTML comments.** The hidden path hint embedded in the page source reflects a common development mistake where internal notes are not removed before deployment.
- **Credential exposure through unauthenticated web content.** The `/pebcak.html` page exposes SSH credentials without any access control, representing a misconfigured web server.
- **Credential reuse and privilege escalation.** The same file contains firewall admin credentials, illustrating how a single disclosure can cascade into full infrastructure compromise.
- **Insufficient network segmentation.** Although pfSense and VyOS provide logical separation, the NAT rules grant unrestricted access to the server's SSH port from any external address.

Lessons Learned

The design and implementation of Lab 1 produced several insights relevant to both the pedagogical and technical aspects of the project:

- **Sequential discovery reinforces methodology.** Structuring the lab as a linear chain — DNS, scan, web, credentials, SSH — gives students a concrete experience of how individual findings build into a full exploitation path, which abstract descriptions of the kill chain cannot replicate.
- **EVE-NG VNC automation requires low-level implementation.** The failure of high-level VNC tools with QEMU nodes necessitated direct protocol implementation, which became a reusable component for all subsequent lab automation.

- **pfSense configuration internals are non-obvious.** The Base64 encoding of Unbound custom options, the PHP-managed service lifecycle, and the post-NAT firewall rule semantics are not documented in the main pfSense guides and required direct inspection of the configuration files to resolve.

6.7 Lab 2: Web Application Vulnerabilities — SYNAPSE Intelligence Portal

6.7.1 Overview and Learning Objectives

The second laboratory focuses on a class of vulnerabilities that consistently appears among the most critical in real-world web applications: injection attacks and client-side scripting abuse. The lab situates these vulnerabilities within a fictional corporate intelligence portal named *SYNAPSE Intelligence Portal*, operated by a company that handles sensitive analyst data. The scenario is designed to be plausible: a public-facing web application with an authenticated back-end, an internal monitoring service, and a multi-tier network architecture.

The attack chain combines four distinct vulnerability classes and requires the student to chain findings sequentially. A successful run produces three intermediate artefacts — a stolen session cookie, a set of database credentials, and remote command execution — before reaching the final flag.

The intended learning outcomes are:

- Understand how stored Cross-Site Scripting (XSS) differs from reflected XSS, and why the stored variant represents a higher-severity threat in authenticated applications.
- Exploit an unsanitised SQL query endpoint to extract data from a back-end database, including credentials stored in plaintext.
- Apply extracted credentials to pivot laterally to a second internal server.